

On this page

ETL process from SAS-EFM to Pulse Run Time

ETL process from SAS-EFM to Pulse Run Time

The goal of Historical Data Load is to make available to Pulse RT event information received in the past before the environment to receive the events in real time is available.

This data will later be used to create / update the profile store - through the execution of the Scheduled Metics Jobs.

ETL will load the Historical Data from the F17 Datasource, which would be provided through CSV files to be loaded to the Events storage database. The diagram below represents the flow to load the historical data load.

Usage

The Historical Data ETL will be configured with the source of the CSV file stored in the storage buckets and the target will be the Pulse events storage database.

The ETL tool for historical data loading is the same tool that we use for the DS Data Migration, however it uses a different configuration:

```
eventstorageetl.numExtractionThreads=25
eventstorageetl.numPublishingThreads=1
eventstorageetl.maxEventsPerSecond=1000
eventstorageetl.pulse.appId=APP_ID
eventstorageetl.pulse.workflowId=WORKFLOW_ID
eventstorageetl.pulse.connectionMode=PAPP
eventstorageetl.pulse.papp=var/tfrb.papp
eventstorageetl.storage=DATABASE
eventstorageetl.database.engine=com.feedzai.commons.sql.abstraction.engine.impl.PostgreSqlEngine
eventstorageetl.database.jdbc=JDBC_CONNECTION
eventstorageetl.database.schema=SCHEMA
eventstorageetl.database.username=USERNAME
eventstorageetl.database.password=ENCRYPTED_PASSWORD
eventstorageetl.transformers=com.feedzai.commons.eventstorage.etl.transform.filter.groovy.UuidKeyTransformer
eventstorageetl.uuidkey.groovy.fieldName=FDZ_UUID
eventstorageetl.uuidkey.groovy.expression=UUID.randomUUID().toString()
eventstorageetl.csv.serializerConfig.schemaDetails.uuidField=lifecycle_id
eventstorageetl.csv.serializerConfig.schemaDetails.timestampField=event_occurred_at
eventstorageetl.csv.serializerConfig.workflowDetails.inputAdapter=INPUT_SERVICE
eventstorageetl.csv.uri=CSV_PATH
```

Copy

Please note the above is kept for historical reasons, but as of Feedzai release 2.4.0, the ETL tool no longer supports PAPP as the connectionMode, nor does it specify a .papp file

Installation with Ansible

Historical ETL is installed as any other tool installed with the ansible toolkit, with playbook `historical-data-loading.yml`.

Execution

The way the tool will be executed in Production is still under review and it will be detailed later. Probably it will be provided an ansible task to launch the tool based on a list of GCS buckets provided via configuration.

The goal of this section is to provide raw information about how the tool is executed and under the hood how it will be called automatically later.

After the installation with ansible, to run the ETL to load Historical Data follow the following steps:

1. go to " /opt/feedzai/historical-etl/etl ".
2. run that command

```
JAVA_OPTS="-Xms8g -Xmx8g ./bin/event-storage-etl -cf ./conf/historical.properties --secret ./conf/secret
```

Copy

Logging and testing

The default logger.xml is provided in `conf/logger.xml` . By default the execution logs will be redirected to:

```
<installation_dir>/log/etl.log
```

Copy

In the previous log file you will be able to find periodic reports about the status of the execution of the data migration about the number of events extracted, and published.

When the extraction finishes a log with the migration result is displayed, similar to the following one:

```
2023-01-23 19:45:17,897 [main] ExecutionMetricFactory.report:330 Report:
[extracted: 8640000/0 @ 4880tps, filtered: 0, published: 8640000/0 @ 4880tps, queued: 0, elapsed
time: 1770532155051 ns]
2023-01-23 19:45:17,898 [main] EventStorageEtlProcessor.runProcess:251 Finished execution
of the ETL process. Time elapsed: 1770532 milliseconds
```

Copy

Exceptions on the historical data load will also be written to the previous log file.

Validations:

- After the execution of the tool the events load to the event storage should be equal to the number of the events in bucket that is being loaded
 - Note the event storage may already have other events, so please check the number before and after the execution to know the number of events loaded
- The number of events extracted and published should be the same (tool logs)
- The log file should not have any exceptions

To check the number of events in event storage:

```
SELECT count(*) FROM <schema>."7b336ee2c9d8ac105d742c45dbde75" where âFDZ_timestampâ >= <start_timestamp_ms> and âFDZ_timestampâ < <end_timestamp_ms>;
```

Copy

Recommendations

Configuration

To achieve the max speed for extraction and publishing we recommend the following configuration:

```
eventstorageetl.numPublishingThreads=25
eventstorageetl.maxEventsPerSecond=5000
eventstorageetl.databaseBatchSize=50
```

Copy

This configuration will ensure the maximum insertion rate while maintaining the stability of the system (processing transactions in real time).

From our tests, with that configuration the ETL should publish the records by rate of "4800 tps", and we run our tests on Compute Engine Instance of type n2-highmem-16, to take advantage of its CPUs capacity.

Historical data ingestion

Taking into account the large dataset expected, in order to reduce as much as possible the impact of possible failures (e.g.: invalid rows in the dataset, or even possible connection issues...) we recommend **against** running ETL in a single execution over the whole data.

Instead, it is better to partition the data and perform multiple executions of ETL with smaller datasets for specific known ranges of time (eg. 1 file per day).

This way, any failure will only affect the known partition and fixing it will be both easier and faster - there's no need to rollback and re-execute the whole dataset.

What to do in Case of Failure during middle of execution?

Execution scenario:

Performing the historical data load for the known range **"9th of September 2022 7 AM till 9th of September 2022 11 AM"**.

Failure scenarios:

1. Connection issues
2. Due to an/many invalid row(s) in the CSV file.

In case 2., the invalid row(s) in the Dataset will need to be fixed.

In both cases we need to retry the same **ETL** execution. Before retrying we need to **delete all** records for that time range, otherwise it could end up with duplicates for rows already inserted previously.

How to delete these records?

We have provided a python Script that we can run by ansible and is parameterized by the time range, so for example if we need to delete all records from **"9th of September 2022 7 AM till 9th of September 2022 11 AM"**.

We can run the ansible role for the time range of the equivalent unix timestamps to be from **1662703200000 to 1662717600000**.

Example for the ansible properties configuration would be like that:

```
events_interval_start: 1662703200000
events_interval_end: 1662717600000
```

Copy